

Universidade Federal de Alfenas (UNIFAL-MG)

Bacharelado em Ciência da Computação

Redes de Computadores

Aluno: João Antonio Siqueira Pascuini

RA: 2024.1.08.028

Relatório Técnico

Implementação do Protocolo Go-Back-N em Java via UDP

1. Introdução

Este relatório descreve a implementação do protocolo Go-Back-N (GBN) desenvolvido para o trabalho final da disciplina de Redes de Computadores. O objetivo foi construir, em Java puro, um par de programas (Emissor e Receptor) capazes de transferir arquivos binários de forma confiável usando apenas sockets UDP, que por natureza não garantem entrega nem ordenação.

Como o UDP não oferece nenhuma garantia, toda a lógica de confiabilidade ficou por conta da aplicação: numeração de pacotes, janela deslizante, confirmações cumulativas (ACKs), temporizador e retransmissão. A implementação seguiu as máquinas de estado do capítulo 3 do Kurose e Ross, e o comportamento foi validado com um mecanismo de perda simulada no lado do Receptor.

2. Decisões de projeto

2.1 Formato do datagrama

Cada datagrama UDP encapsula um segmento com cabeçalho próprio de 11 bytes, seguido de até 1024 bytes de payload. Os campos são: tipo (1 byte, indicando DATA, ACK, HANDSHAKE ou FIN), número de sequência (4 bytes), número de ACK (4 bytes) e tamanho dos dados (2 bytes). A montagem e a leitura do cabeçalho foram feitas com a classe ByteBuffer da biblioteca padrão, que garante que os campos sejam escritos e lidos sempre na mesma ordem e com o mesmo tamanho, evitando problemas de portabilidade entre as duas pontas.

Essa lógica foi centralizada em uma classe iúnica, Datagrama, compartilhada pelo Emissor e pelo Receptor. Funcionando como o contrato entre os dois módulos: qualquer mudança no formato do pacote acontece em um lugar só.

2.2 Handshake

Antes da transferência, o Emissor envia um pacote de controle do tipo HANDSHAKE contendo a probabilidade de perda e o caminho absoluto onde o arquivo deve ser salvo, em formato de texto separado por barra vertical (por exemplo, 0.1|C:\tmp\arquivo.pdf). O Receptor extrai esses

parâmetros, responde com um ACK e só então a transferência de dados começa. Essa foi uma decisão de projeto simples que evitou a necessidade de configurar o Receptor manualmente a cada execução.

2.3 Janela deslizante e temporizador

O Emissor mantém as variáveis base (pacote mais antigo não confirmado) e nextSeqNum (próximo a enviar), e só transmite enquanto nextSeqNum estiver dentro da janela de tamanho N. O temporizador foi implementado com um ScheduledExecutorService: existe um único timer, associado ao pacote da base, exatamente como na FSM do livro. Quando um ACK cumulativo avança a base, o timer é cancelado e reiniciado se ainda houver pacotes pendentes. Quando o timeout de 500 ms expira, todos os pacotes de base até nextSeqNum menos 1 são retransmitidos.

2.4 Concorrência

O Emissor usa duas threads: a principal envia pacotes conforme a janela abre, e uma segunda fica bloqueada no receive() aguardando ACKs. Como as duas mexem nas mesmas variáveis (base e nextSeqNum), protegi as regiões críticas com blocos synchronized sobre um objeto de lock compartilhado, usando wait() e notifyAll() para a thread principal dormir quando a janela está cheia e acordar quando um ACK chega ou um timeout dispara. Os pacotes já enviados ficam guardados em memória numa lista, o que permite retransmitir sem reler o arquivo do disco.

2.5 Receptor

O Receptor implementa a FSM de estado único do GBN: aceita apenas o pacote cujo número de sequência é igual a expectedSeqNum, grava o payload no arquivo de saída e responde com ACK. Pacotes fora de ordem são descartados e o ACK do último pacote aceito é reenviado. A simulação de perda sorteia um número entre 0 e 1 para cada pacote em ordem recebido; se o valor for menor que a probabilidade configurada, o pacote é descartado silenciosamente, sem ACK, forçando o timeout no Emissor. Pacotes fora de ordem não entram nessa contagem, conforme exigido no enunciado.

3. Dificuldades encontradas

- O bug mais difícil de encontrar foi um erro de digitação nas constantes de tipo de pacote: TIPO_DATA, TIPO_ACK, TIPO_HANDSHAKE e TIPO_FIN acabaram todas definidas com valor 0. Com isso, o primeiro pacote de dados era interpretado como FIN e o Receptor encerrava a transferência imediatamente, com zero pacotes salvos, enquanto o Emissor ficava em loop de timeout. Só foi possível localizar o problema adicionando prints de depuração no Receptor mostrando o tipo e a sequência de cada pacote recebido.
- A sincronização entre as duas threads do Emissor exigiu cuidado. Nas primeiras versões, era fácil a thread principal ficar dormindo para sempre ou o timer ser reiniciado em momento errado. A solução foi concentrar toda a manipulação de base e nextSeqNum dentro de blocos synchronized e usar notifyAll() em todos os pontos que mudam o estado da janela (chegada de ACK e timeout).

- Um detalhe prático: caminhos de arquivo com espaço no nome quebram a leitura dos argumentos de linha de comando, já que cada palavra vira um argumento separado. A solução adotada nos testes foi usar caminhos sem espaços.

4. Testes e resultados

Os testes foram feitos em uma única máquina, usando o endereço de loopback (127.0.0.1), com um arquivo PDF de 1.660.746 bytes, que dividido em segmentos de 1024 bytes gera 1622 pacotes de dados.

O primeiro teste foi com probabilidade de perda 0.0 para validar a transferência básica. Em seguida, os testes principais usaram perda de 10%, variando o tamanho da janela N. A integridade do arquivo recebido foi verificada com comparação byte a byte (comando `fc /b` do Windows) entre o arquivo original e o recebido, sem diferenças em todos os casos.

4.1 Variação do tamanho da janela (perda de 10%)

Janela (N)	Tempo total (s)	Retransmissões	Pacotes fora de ordem	Perda efetiva
1	81,51	161	0	9,03%
4	90,40	711	532	9,94%
8	82,80	1305	1141	9,18%
16	83,89	2636	2470	9,28%

4.2 Análise

A taxa de perda efetiva ficou entre 9,03% e 9,94% em todas as execuções, muito próxima dos 10% configurados. Isso confirma o comportamento esperado pela Lei dos Grandes Números: com 1622 pacotes ou mais transferidos (contando retransmissões), a frequência de descartes converge para a probabilidade sorteada.

O resultado mais interessante está na relação entre o tamanho da janela e o número de retransmissões. Com N igual a 1, cada perda causou a retransmissão de exatamente 1 pacote, e por isso o total de retransmissões (161) é igual ao total de perdas simuladas. Conforme N cresce, cada timeout retransmite a janela inteira, mesmo que apenas um pacote tenha se perdido: com N igual a 16, as 166 perdas geraram 2636 retransmissões, quase 16 vezes mais tráfego redundante. Os pacotes fora de ordem crescem na mesma proporção, porque as retransmissões da janela chegam ao Receptor depois que ele já reenviou o ACK duplicado.

Já o tempo total praticamente não variou (entre 81 e 90 segundos). Isso acontece porque o teste roda em loopback, onde o tempo de ida e volta é praticamente zero: o custo dominante é o próprio timeout de 500 ms que cada perda provoca. Com cerca de 165 perdas por execução, só de espera de timeout já são aproximadamente 82 segundos, o que explica o tempo observado em todos os cenários. Em uma rede real, com atraso de propagação relevante, janelas maiores tenderiam a

melhorar o throughput por manterem mais pacotes em trânsito; em loopback esse benefício não aparece.

Esse comportamento também evidencia a principal limitação do Go-Back-N em cenários com perda: o desperdício de banda com retransmissões desnecessárias, exatamente o problema que o Selective Repeat resolve ao confirmar e armazenar pacotes fora de ordem individualmente.

5. Conclusão

A implementação atendeu aos requisitos do trabalho: a transferência de arquivos binários funcionou corretamente mesmo com 10% de perda simulada, as estatísticas coletadas confirmaram o comportamento teórico do protocolo e a variação do tamanho da janela permitiu observar na prática o custo das retransmissões em bloco do Go-Back-N. Além do resultado em si, o desenvolvimento deixou claro na prática, pois protocolos reais como o TCP evoluíram para mecanismos mais seletivos de retransmissão e controle.

Referências

KUROSE, James F.; ROSS, Keith W. Redes de Computadores: Uma Abordagem Top-Down. 8. ed. São Paulo: Pearson, 2021. Capítulo 3.

TANENBAUM, Andrew S.; WETHERALL, David. Redes de Computadores. 5. ed. São Paulo: Pearson, 2011. Capítulo 3.